

Automata

Временско ограничење: 2 секунде

Меморијско ограничење: 1024 MiB

Дато је поље које чини N ћелија поређаних у реду, нумерисаних од 0 до $N - 1$ с лева на десно. Свака ћелија има јединствену висину: висина ћелије i је p_i , а низ $p_0, p_1, \dots, p_{N-1}$ је пермутација бројева $0, 1, \dots, N - 1$.

Две ћелије са **индексима** i и j су *блиске* ако и само ако је $|i - j| \leq 1$. Посебно, свака ћелија је блиска сама себи.

Робот је на пољу, на почетку постављен на некој ћелији. Робот прима команде, које су једног од наредна два типа:

- **MAX**: међу свим ћелијама блиским роботовој тренутној ћелији, робот прелази на јединствену ћелију која има **максималну** висину;
- **MIN**: међу свим ћелијама блиским роботовој тренутној ћелији, робот прелази на јединствену ћелију која има **минималну** висину.

Како је свака ћелија блиска сама себи, команда може да остави робота на истој ћелији на којој се налазио.

Програм је коначан низ команди, где је свака команда или **MAX** или **MIN**. Ако робот креће из ћелије X и прати програм S , на крају ће се увек наћи у некој јединствено одређеној ћелији коју означавамо са $\text{result}(S, X)$.

Биће ти постављено Q упита, сваки од њих ће садржати скуп почетних ћелија величине K_i ($0 \leq i < Q$). Прецизније, i -ти упит ће се извршавати над ћелијама $x_0, x_1, \dots, x_{K_i-1}$; робот ће бити постављен на неку од њих, али није унапред одређено коју. За сваки упит треба да одредиш да ли постоји програм који доводи робота у исту крајњу ћелију, без обзира на то која од ћелија из x је изабрана за почетну.

Формално, треба да одредиш да ли постоји програм S такав да је:

$$\text{result}(S, x_0) = \text{result}(S, x_1) = \dots = \text{result}(S, x_{K_i-1}).$$

Пермутација p је **иста** за све упите.

Примети да не треба да конструишеш описани програм – само да одредиш да ли такав постоји.

Детаљи имплементације

Прва функција коју мораш да имплементираш је:

```
void initialize(std::vector<int> p)
```

- p : пермутација бројева од 0 до $N - 1$.

Друга функција коју мораш да имплементираш је:

```
bool exists_program(std::vector<int> x)
```

- x : низ у којем се наводе ћелије у строго растућем поретку за дати упит.

Функција `initialize` се позива тачно једном, пре свих позива функције `exists_program`.

Функција `exists_program` се позива Q пута, једном за сваки упит. Она мора да врати `true` ако постоји програм након чијег извршавања робот завршава у истој ћелији без обзира на то из које од почетних ћелија је кренуо, и `false` иначе.

Ограничења

- $3 \leq N \leq 2 \cdot 10^5$
- $1 \leq Q \leq 5 \cdot 10^5$
- $p_0, p_1, \dots, p_{N-1}$ је пермутација бројева $0, 1, \dots, N - 1$
- $2 \leq K_i \leq N$
- $0 \leq x_0 < x_1 < \dots < x_{K_i-1} < N$ за сваки упит
- Збир K_i за свих Q упита није већи од 10^6 .

Примери

Пример улаза 1	Пример излаза 1
3 3 0 2 1 2 0 2 3 0 1 2 2 1 2	111

Пример улаза 2	Пример излаза 2
7 3 0 4 2 1 3 5 6 3 0 1 3 3 1 3 4 2 3 6	001

Објашњење примера 1

У овом примеру је $p = [0, 2, 1]$. У другом упиту, робот може да крене из ћелије 0, ћелије 1 или ћелије 2. Размотримо програм `[MAX]`.

- Када робот крене из ћелије 0: ћелије блиске ћелији 0 су ћелије са индексима 0 и 1. Када робот изврши команду `MAX`, како је $p_0 < p_1$, он се помера на ћелију 1.
- Када робот крене из ћелије 1: ћелије блиске ћелији 1 су ћелије са индексима 0, 1 и 2. Када робот изврши команду `MAX`, како је $p_0 < p_1$ и $p_1 > p_2$, он остаје на ћелији 1.
- Када робот крене из ћелије 2: ћелије блиске ћелији 2 су ћелије са индексима 1 и 2. Када робот изврши команду `MAX`, како је $p_1 > p_2$, он се помера на ћелију 1.

Дакле, постоји програм након чијег извршавања робот заврши у ћелији 1 крећући из све три почетне ћелије, па је одговор за овај упит `true`. Други програми такође раде, као на пример `[MIN, MAX]`, `[MIN, MIN, MAX, MIN, MAX, MIN]` и `[MAX, MIN, MAX]`.

Објашњење примера 2

У овом примеру је $p = [0, 4, 2, 1, 3, 5, 6]$.

За прва два упита може се показати да не постоји програм за који би робот завршио у истој ћелији крећући из свих почетних ћелија, па је одговор за оба `false`.

Размотримо последњи упит, у ком робот може да крене из ћелије 3 или ћелије 6, и размотримо програм `[MAX, MAX, MAX]`.

- Када робот крене из ћелије 3: ћелије блиске ћелији 3 су ћелије са индексима 2, 3 и 4. Како је $p_3 < p_4$ и $p_2 < p_4$, робот се помера на ћелију 4. Пратећи наредне две команде на исти начин, робот завршава на ћелији 6.
- Када робот крене из ћелије 6, он остаје на ћелији 6 током извршења програма и остаје ту до краја.

Дакле, одговор за овај упит је `true`. Програм `[MIN, MAX, MAX, MAX]` такође ради. Међутим, други програми не раде нужно. На пример, ако је програм `[MAX, MAX]`, робот завршава у ћелији 5 када крене из ћелије 3, али завршава у ћелији 6 када крене из ћелије 6.

Подзадаци

Подзадатак	Поени	N	Q	K_i	Додатна ограничења
0	0	–	–	–	Примери.
1	3	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$= 2$	Ако је одговор на упит позитиван, тада постоји валидни програм са тачно 1 командом.
2	7				Ако је одговор на упит позитиван, тада постоји валидни програм са највише 5 команди.
3	9				–
4	17				–
5	7				$x_1 = x_0 + 1$ у сваком упиту.
6	8	$\leq 5\ 000$	$\leq 5 \cdot 10^5$	$\leq N$	–
7	13	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$		Пермутација је прво растућа, потом опадајућа, а онда опет растућа. Формално, постоје два цела броја a, b таква да је $0 < a < b < N - 1$ и $p_0 < p_1 < \dots < p_a > p_{a+1} > \dots > p_b < p_{b+1} < \dots < p_{N-1}$.
8	13				$p_0 < p_1 > p_2 < p_3 > \dots < p_{N-1}$ ако је N парно, или $p_0 < p_1 > p_2 < p_3 > \dots > p_{N-1}$ ако је N непарно.
9	23				–

Пример грејдера

Формат улаза је следећи:

- линија 1: два цела броја – вредности N и Q ;
- линија 2: N целих бројева $p_0, p_1, \dots, p_{N-1}$, где је p_i висина i -те ћелије;
- линија $3 + i$: почетне ћелије дате за i -ти упит – један цео број K_i , којег у истој линији прати K_i целих бројева $x_0, x_1, \dots, x_{K_i-1}$, могуће почетне ћелије робота.

Формат излаза је следећи:

- линија 1: низ битова дужине Q , где је i -ти бит 1 ако је одговор на i -ти упит true, и 0 иначе.